

→ Indexing Techniques

→ File Organization & Indexing:

- The file is a collection of records.
- Using the primary key, we can access the records.
- File Organization is a logical relationship among various records.
It defines how file records are mapped onto disk blocks.

Types of file organization:

There are mainly 6 types of file organization

1. Sequential File Organization
2. Heap File Organization
3. Hash File Organization
4. Indexed Sequential access method (ISAM)
5. B+ Tree file Organization
6. Cluster file Organization

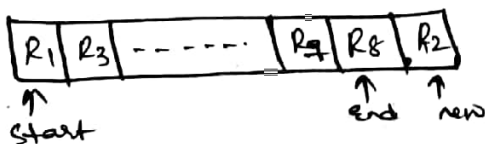
Sequential File Organization:

In this type files are stored sequentially. It implemented in two ways.

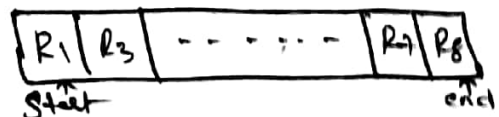
1. File file method: In this we store the record in a sequence i.e one after another. In case of updating, deleting of any record, then the record will be searched in the memory blocks. when it is found then delete.



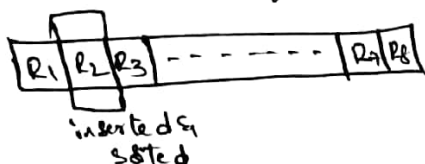
Insertion: we want to insert R_2 (new record) in the sequence. then it will be placed at the end of the file.



2. Sorted file method: The new record is inserted at the file end, & then it will sort the sequence in ascending & descending order. Sorting of records is based on primary key & another key.



Insertion: we want to insert new record R₂. It will be inserted at the end of file & then it will sort



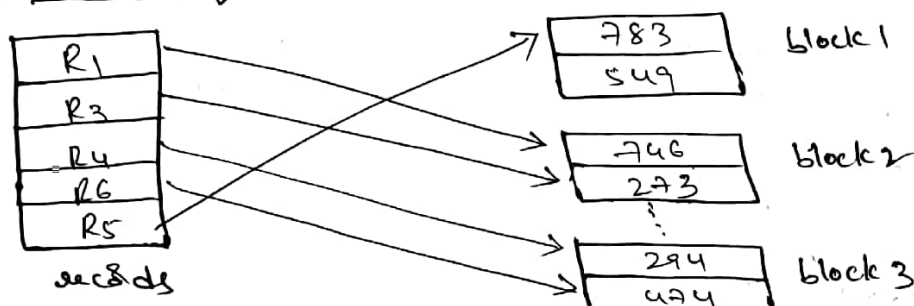
- Adv:
1. It is Simple
 2. files can be easily stored
 3. fast & efficient of huge amount of data

dis. Adv:

1. waste time, we can't jump on a particular record
2. sorted file method takes more time & space for sorting

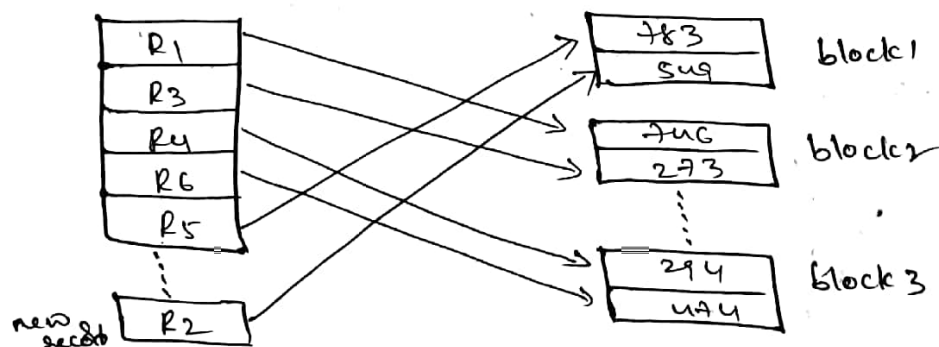
2. Heap File Organization:

- In this technique, when the records are inserted at the files end. It doesn't require the sorting & ordering of records.
- when block is full, the new record is stored in some other block



Insertion:

If we want to insert new record R₂ in a heap. block 3 is full then it will be inserted in any of the database block.

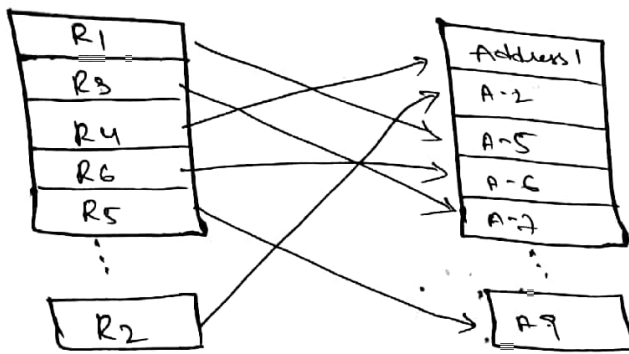


- Adv:
1. good for bulk insertions
 2. faster retrieving of records

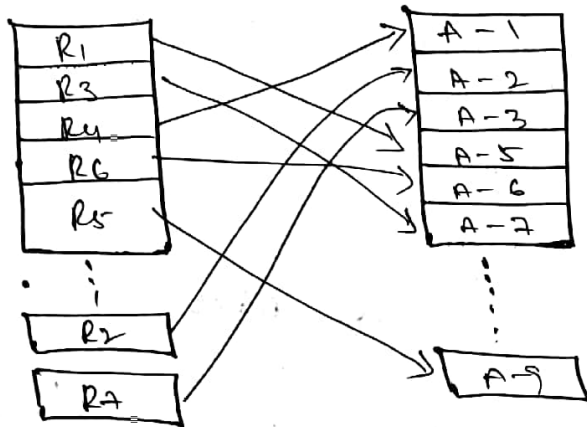
dis. Adv: 1. Insufficient for large data bases.

3. Hash File Organization:

- It uses hash function. output determines the location of disk blocks where the records are to be placed
- by using hash functions only we placed (or) insert data in memory blocks

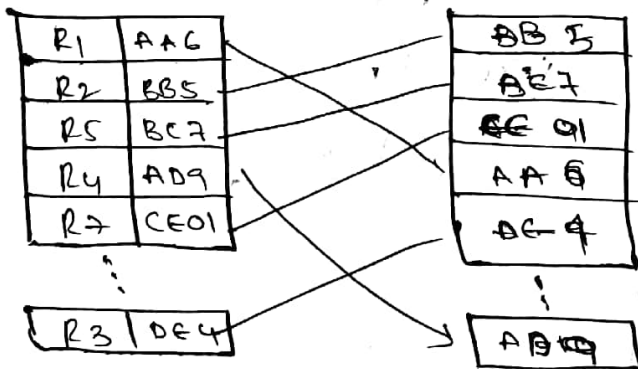


When new record inserted then address is generated using hash key & inserted in that address...



4. Indexed Sequential Access Method (ISAM)

- It is advanced sequential file organization.
- In this method records are stored in the file using the primary key. The index value is generated for each key & mapped with record. This index contains the address of the record.



based on index we can delete, update the data

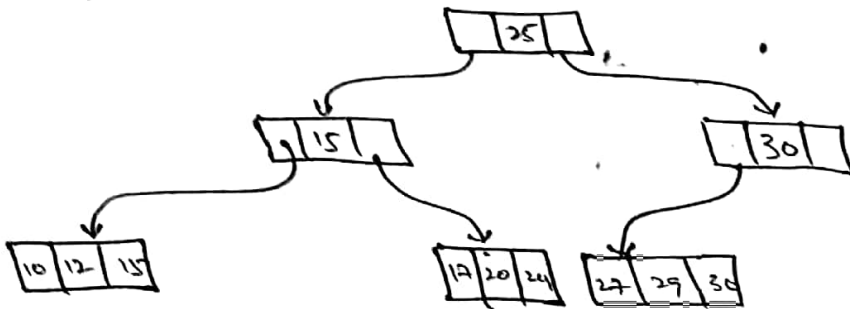
Adv: 1. Each record has the address so quick search

Dis. Adv: 1. It requires extra space to store index.

5. B+ File Organization:

- It is advanced method of ISAM.
- It uses tree-like structure to store records.
- The B+ tree is similar to binary search tree but it can have more than two children.

- All the records are stored in leaf nodes only.
- Intermediate nodes act as a pointer to the leaf nodes.
- They do not contain any record.



- There is one root node 25. left nodes the < 25 & right nodes are > 25
- Searching for any record is easier, as all the leaf nodes are balanced.
- Single path only we can access data.

Adv:

1. Searching becomes Easy
2. Size of tree has no restriction
3. It is balanced, any insert/update/delete does not affect the performance of tree

dis Adv:

1. It is insufficient for the static methods.

6. cluster file organization

- Two or more records are stored in same file is known as cluster. key attributes are used to map these records.
- It is used to join the tables

Emp

Id	Name	Address	Dep-id
1	A	GNT	14
2	B	GNT	12
3	C	GNT	15
4	D	H4D	19

Dept

Dep-id	name
10	math
11	physics
12	chemistry
14	english

cluster key
↓

Dep-id	name	Id	name	Address
10	math	1	A	
11	physics	2	B	
12	chemis	2	B	GNT
14	English	1	A	GNT
15		3	C	GNT
19		4	D	H4D

Adv: - used when request for joining the tables with same joining condition

dis Adv:

1. low performance of very large database
2. If joining condition change then this method can't use.

→ Indexing:

The index is a type of data structure. It is used to locate & access the data in a database table 'quickly'.

Search Key	Data Reference
------------	----------------

Index Structure

- 1st column of index contain a primary key or candidate key. Every primary key is unique & sorted using this key only so we can access easily.

- 2nd column is data reference. It contains Set of pointers holding the address of the blocks where the value of the particular key can be found.

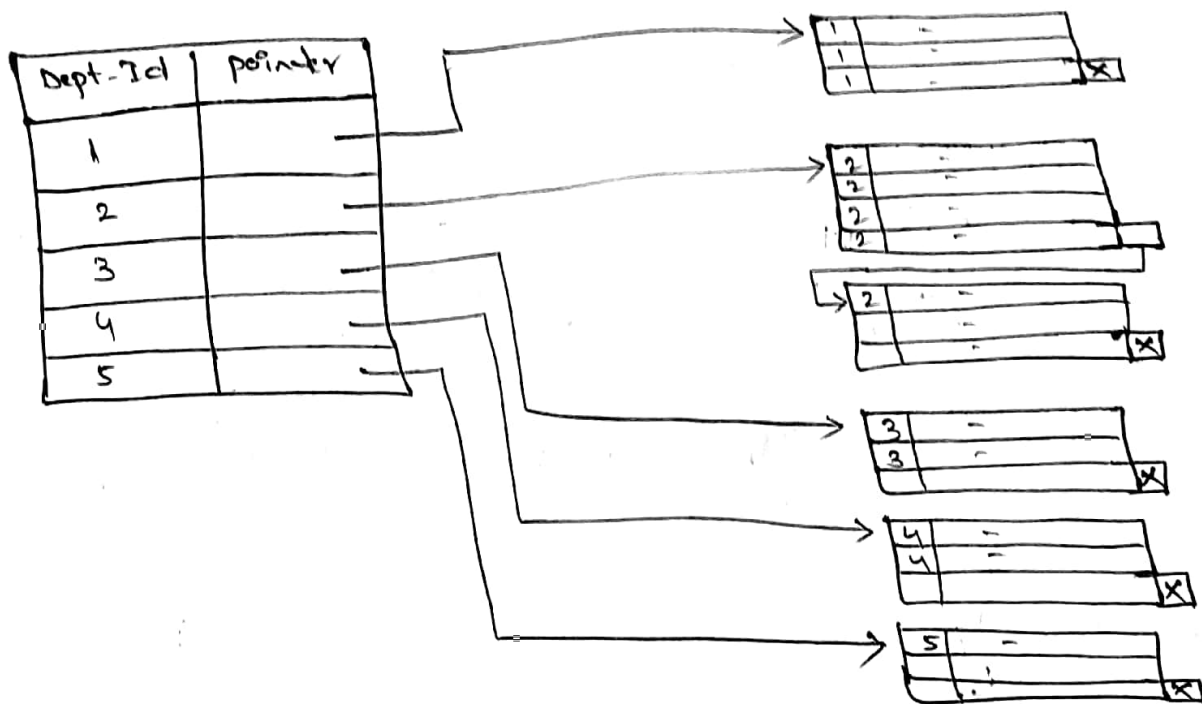
Indexing methods:

1. Ordered Indices
2. primary Index ← Dense Index
← Sparse Index
3. clustering Index
4. Secondary Index.

→ cluster Indexes:

- A cluster index can be defined as an ordered file. Some times the index is created on non-primary key columns, which may not be unique for each record.
- In this case to identify the record faster, we will group two or more columns to get the unique value & create index out of them. This method is called clustering index.
- The records which have similar characteristics are grouped and indexes are created for these group.

Ex:-
A company contains several employees in each department. If we use cluster indexing then we will group the employees of same departments in a single cluster. & Index pointer points to the employees details. Here Dept-Id is a non-unique key.



→ Primary & Secondary Indexes

Primary Index:

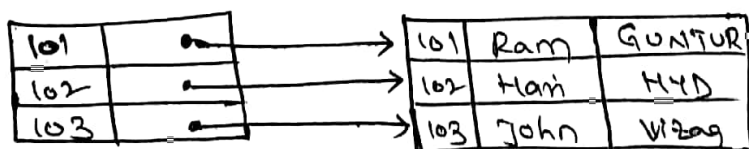
- If the index is created on the basis of primary key of the table, then it is known as primary indexing.
- primary keys are in sorted order so the performance of searching is efficient.
- It is classified into two types:

1. Dense Index

2. Sparse Index

Dense Index: In this the number of records in the index table is same as the number of records in the main table.

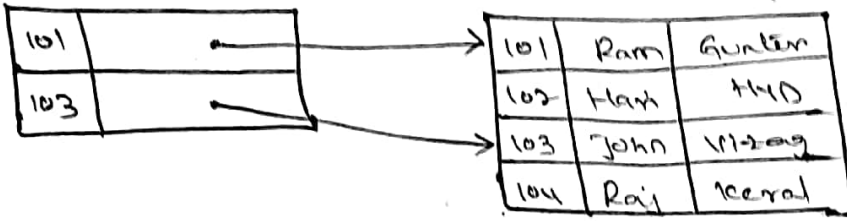
- It needs more space to store index record itself.
- The index records have the search key & a pointer to the actual record on the disk.



Sparse Index: In this index record appears only for a few items.

Each item points to a block.

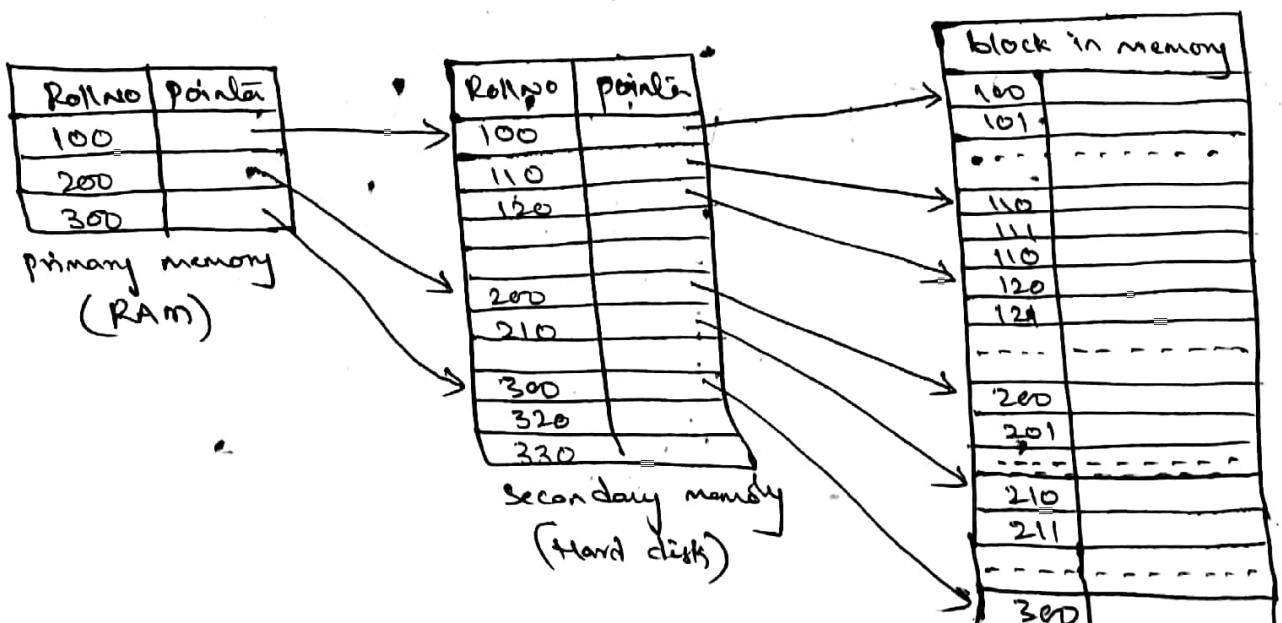
- Instead of pointing to each record in the main table, the index points to the records in the main table in a gap.



Secondary Index

In the Sparse Indexing, the size of the table grows, the size of mapping also grows. These mappings are usually kept in the primary memory so that address fetch should be faster. If mapping size grows then fetching the address itself becomes slower. In this case sparse index will not be efficient. To overcome this problem, Secondary indexing is introduced.

In Secondary indexing to reduce the size of mapping another level of indexing is introduced. In this, the huge range for the columns is selected initially so that mapping size of the first level becomes small. Then each range is further divided into smaller ranges. 1st level mapping is stored in the primary memory, so that address fetch is faster. 2nd level mapping & actual data are stored in the secondary memory (hard disk).



Ex:- If we want to find the record of roll no 111, then it will search the highest entry, which is smaller than or equal to 111 in primary memory. It will get 100 at this level.

- Then in the secondary memory, again it search for highest then get address 110, it goes to the block & start searching each record till it gets 111.
- So here 2 levels we will perform searches.

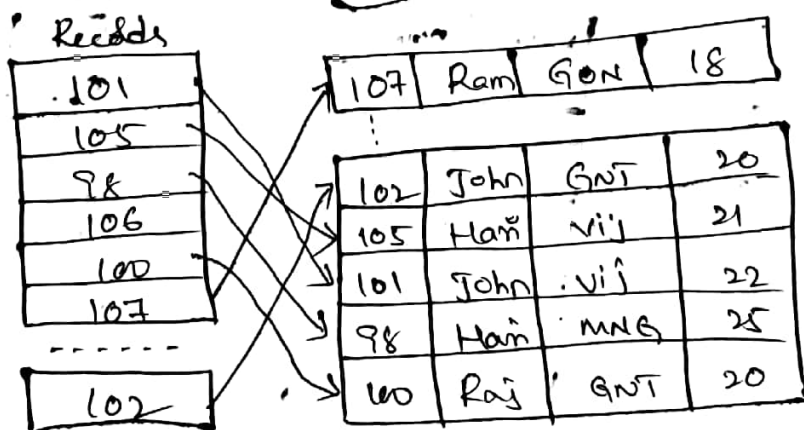
→ Hash Based Indexing

Hashing technique is used to calculate the direct location of a data record on the disk without using index structure.

In this technique, data is stored at the memory, whose address is generated by using the hash function. This memory location is called as data block or data bucket.

In this, a hash function can choose any of the column value to generate the address. Most of the time, the hash function uses the primary key to generate the address of the data block. Some times primary itself consider the address of the data block.

Ex:-



The above diagram shows data blocks addresses same as primary key value. Suppose we have use mod function to determine the address of the data blocks.

$$\text{Mod}(\text{key}) = \text{key} \% \text{no. of records}$$

$$\text{mod}(101) = 101 \% 7 = 3$$

$$\text{mod}(105) = 105 \% 7 = 0$$

$$\text{mod}(98) = 98 \% 7 = 0$$

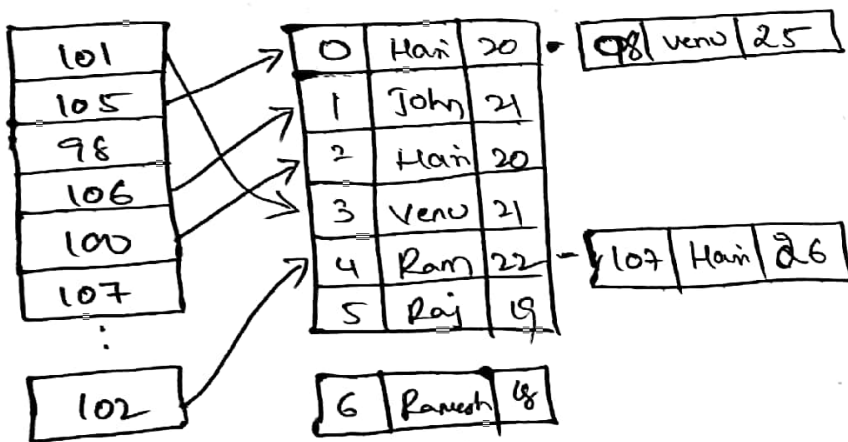
$$\text{mod}(106) = 106 \% 7 = 1$$

$$\text{mod}(100) = 100 \% 7 = 2$$

$$\text{mod}(107) = 107 \% 7 = 2$$

$$\text{mod}(102) = 102 \% 7 = 4$$

If we apply hash function generates 3, 0, 0, 1, 2, 2, 4 are the locations



Types:

2 types of hashing

1. Static hashing

2. Dynamic hashing

Static hashing:

In static hashing, the resultant data bucket address will always be the same. i.e. if we generate an address of 101 using the hash function $\text{mod}(7)$ it will always result in same bucket address 4. there will be no change in the bucket address.

Operations

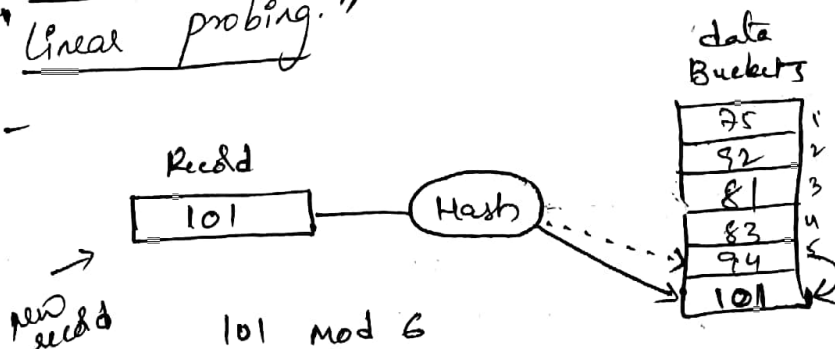
1. Searching : hash function is used to search
2. Inserting : Based on hash function generate hash key in that location we will insert ^{record} ~~data~~
3. Deleting : 1st we will search & after delete that address
4. updating : 1st we will search using hash function & then updating record

- If we want insert a new record but that address having already some record then that situation is called bucket overflow. To overcome this problem 2 mechanisms are used:

1. Open hashing
2. Close Hashing

Open hashing: When a hash function generates an address at which data is already exist, then next bucket will be allocated to it. This mechanism is called as "linear probing".

Ex:-



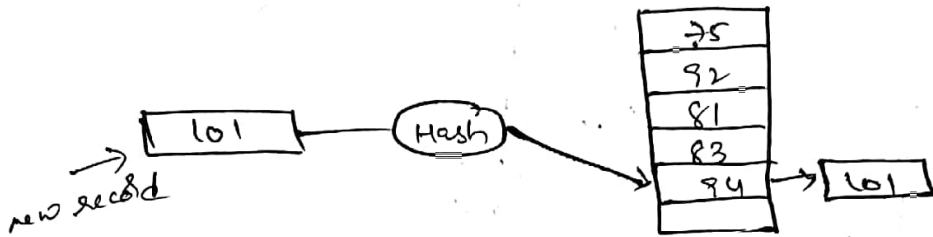
$$\begin{array}{r} 6 \overline{) 101 (16 \\ \underline{60} \\ 41 \\ \underline{36} \\ 5 \end{array} \quad \begin{array}{r} 6 \overline{) 81 (13 \\ \underline{48} \\ 33 \\ \underline{24} \\ 9 \end{array}$$

here 101 address is 5 but there is already 94 exists
so we will allocate next location.

Close Hashing:

when buckets are full, then a new data bucket is allocated for the same hash result & is linked after the previous one. This mechanism is known as "overflow chaining."

Ex:-



here 101 is allocated in 5th location & linked to 94

Dynamic Hashing

Dynamic hashing method is used to overcome the problems of static hashing like bucket overflow. In this technique we will use binary digit mechanism. Least significant bit is used as index.

Ex:- 9, 4, 3, 5, 7, 8, 6

00	4, 8	B0
01	9, 5	B1
10	6	B2
11	3, 7	B3

4 & 8 least Significant bits are 00
 9 & 5 least Significant bits are 01
 6 least Significant bits are 10
 3 & 7 least Significant bits are 11

8	4	2	1	
0	0	0	0	- 0
0	0	0	1	- 1
0	0	1	0	- 2
0	0	1	1	- 3
0	1	0	0	- 4
0	1	0	1	- 5
0	1	1	0	- 6
0	1	1	1	- 7
1	0	0	0	- 8
1	0	0	1	- 9
1	0	1	0	- 10
1	0	1	1	- 11
1	1	0	0	- 12
1	1	0	1	- 13
1	1	1	0	- 14
1	1	1	1	- 15

If bucket is full then create 3 digit data records i.e. if we add '13' in that bucket then least significant bits of 13 are 01. but B1 is already full then we will take the data records are

000	
001	
010	
011	
100	
101	
110	
111	

4,8	B0
9,15	B1
6	B2
3,7	B3
1	B4
	B5
13	B6
	B7

13
1101

here '01' already full so we will check 3 digits of 13 i.e. 101 - now we will insert 13 in 101 bucket

Adv:

In this method, the performance does not decrease if data grows it simply increases the size of the memory.

Dis. Adv:

If the data size increases then the bucket size is also increased, here the problem is data will change as buckets grow & shrink.

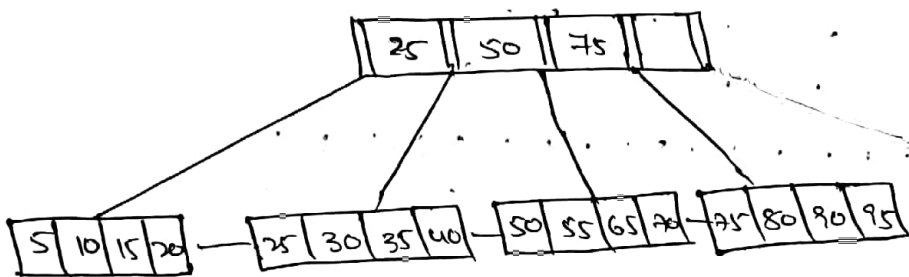
- here also bucket overflow situation will also occur. But it might take little time to reach this situation than static hashing.

→ Tree Base Indexing

Here we use B+ Tree.

- The B+ Tree is a balanced 'Binary' Search Tree. It follows a multi-level index format. In this, leaf nodes denotes actual data pointers.
- It maintain all leaf nodes at the same height.
- In the B+ Tree, the leaf nodes are linked using a linked list. Therefore, a B+ tree can support Random access & Sequential access.

Ex:- 25, 15, 20, 5, 10, 35, 40, 50, 30, 55, 50, 70, 65, 75, 95, 80, 90



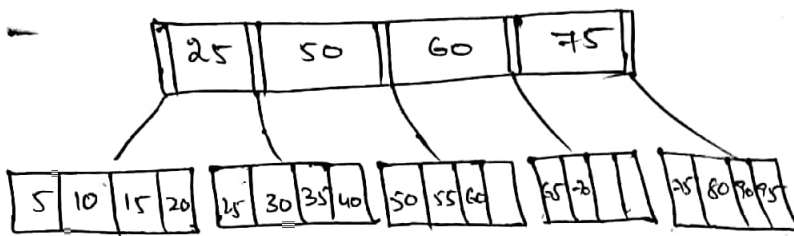
Searching:

Suppose we have to search 55 in the above B+ tree. first we will check root node near to 55. So the 55 node is < 75 & > 50 so we will go to 3rd leaf node.

Insertion:

Suppose we want to insert 60 in the above tree. It will go to 3rd leaf node after 55. The leaf node is already full. so we can't insert 60 there.

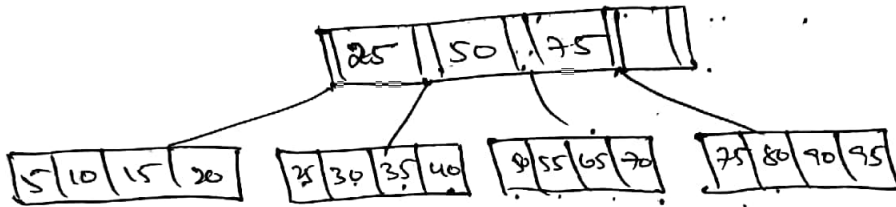
In this case, we have to split the leaf node so that it can be inserted into tree without affecting balance & order.



Deletion:

Suppose we want to delete 60 from the tree we have to remove 60 from the index root node then tree will become imbalanced, so we need to modify it to have a balanced tree.

After deleting node 60 then the re-arranging nodes as



→ comparison of file organization

File organization 6 techniques are have to write here.

i.e

1. Sequential file organization
2. Heap file organization
3. Hash file organization
4. Indexed Sequential Access method (ISAM)
5. B+ Tree organization
6. cluster file organization.